CSE 211 — Theory of Computation

Pushdown Automata — Design

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Hakim Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — where the PDA questions are tracked

Lesson 22 — Why a Stack; the PDA and Its Computations — 6-tuple, labels a, b → c, configurations, 0^n1^n

Lesson 23 — Nondeterminism and Design Patterns — guess the midpoint, guess the branch, design checklist

Lesson 24 — Ten Worked PDA Designs and TF Problems — count/match, store/reverse, i = j or i = k, nested blocks

Quick Notes — PDA Design — revision bullets + key diagrams

> **NOTE** Slide numbers refer to the PDF pages of lec_08_pda.pdf (71 pages). Sipser notation is used: a transition 'a, b → c' reads a, pops b, pushes c (ε = nothing). Every PDA in this note was run by a simulator on all strings up to length 7 (4–5 for larger alphabets) against the language definition. Practice provenance: Actual = taken from a TF paper; Adapted = TF question with changed details; Constructed = new question in TF style.

Exam Map — What the TF Papers Ask

Session	PDA questions (all tracked in Atif Sir's tracker, tab 2)
2023-24	Q8 PDA for {a^n b^m c^m d^n | n ≥ 1, m ≥ 1} (15)
2022-23	Q8 PDA for {a^i b^j c^k | i = j or i = k} (15)
2021-22	Q7 PDA for ARITH = legal arithmetic expressions over {a,+,∗,(,)} (15)
2020 (Jan term)	Q8 PDA for {a^i b^j c^k | i = j or i = k} (15)
2019-20	Q8 PDA for ARITH = legal arithmetic expressions over {a,+,∗,(,)} (15) · Q10(a) Final-state PDAs and empty-stack PDAs accept the same languages (8)
2017-18	Q8 PDA for {a^n b^n c^m d^m | n ≥ 1, m ≥ 1} (15)
2016-17	Q6(a) PDA for {0^n 1^m | n ≤ m ≤ 2n}; show it accepts 00111 and rejects 0011111 (15) · Q7(b) PDA for {ww^R | w ∈ {0,1}∗} (10)
2015-16	Q5(a) Draw the transition diagram of PDA P = ({q,p},{0,1},{Z0,X},δ,q,Z0,{p}) (8⅓) · Q5(b) All reachable IDs of that PDA from (q,w,Z0) for 0011, 0101, 1010 (15) · Q6(a) PDA for {w ∈ {a,b}∗ : equal numbers of a's and b's} (13) · Q6(b) From PF (final state) build PN (empty stack) for the same L — definitions of both (10⅓)
2014-15	Q6(c) PDA for {wcw^R | w ∈ {a,b}∗}; trace bbcbb by configurations (9⅓)
2013-14	Q1(b) PDA for {a^i b^j c^k : i = j or j = k} (7)
2012-13	Q4(a) PDA for {a^i b^j c^k : i + j = k} (8) · Q4(c) PDA for odd-length strings over {a,b} with middle symbol a (8⅓)
2011-12	Q3(c) PDA for odd-length strings over {a,b} with middle symbol a (7) · Q4(a) PDA for {a^i b^j c^k : i + j = k} (7)
2010-11	Q13(a) Formally describe a PDA for {w ∈ {0,1}∗ | w = w^R} (8) · Q13(b) Convert the PDA of 13(a) into an equivalent CFG (7⅓) · Q13(c) Run the given PDA (q1…q4, $ marker) on 1001 and 0101; what does it do? (8)
2009-10	Q10(a) Formal PDA definition; PDAs for {0^n1^n | n ≥ 1}, equal 0s and 1s, palindromes over {0,1} (14) · Q10(b) PDA P({p,q},…,δ: (q,0,Z)→XZ, …): (i) transition diagram, (ii) convert to an empty-stack PDA for the same language (9⅓) · Q11(b) Convert PDA δ(q,1,Z)={(q,XZ)}, δ(q,0,X)={(p,X)}, δ(p,1,X)={(p,ε)} to a CFG (13)
2008-09	Q1(i) PDA for {a^n b^2n | n > 0} or prove impossible (10)
2007-08	Q5 PDA for {a^i c^k (bc)^i | i, k ≥ 0} (10) · Q6 Justify: L has a PDA accepting by final state iff it has one accepting by empty stack (10)
2006-07	Q8 Define PDA with an example; what is an instantaneous description (ID)? (10) · Q9 Transition diagram of a PDA accepting palindromes of all lengths over {0,1} (10)

EXAM FOCUS PDA topics are taught by BOTH teachers. In every past paper the PDA questions sat in the Turing-machine section, so the tracker lists them under Atif Sir's part (PDA tab) and this note has no tracker rows of its own — it is the design workshop for them. The ★★★ design questions are i = j or i = k (2022-23, Jan-2020; 2013-14 had j = k), nested/sequential blocks a^nb^mc^md^n and a^nb^nc^md^m (2023-24, 2017-18), and ARITH (2021-22, 2019-20). Lesson 24 solves the first two in Hakim Sir's notation; ARITH, traces and conversions are in the Atif PDA note.

Lesson 22 — Why a Stack; the PDA and Its Computations

Slides 1–28

1. Why a Stack? (sl. 4–10)

A = {0^n1^n} needs unbounded memory: a DFA has finitely many states and cannot store n. A stack (last in, first out; push and pop only at the top) can push one marker per 0 and pop one per 1, detecting mismatch or wrong order. The stack height records #0 − #1 (sl. 8). Bad strings fail: 001 ends with a 0 left on the stack; 011 needs a pop from an empty stack; 0101 has a 0 after the 1-phase began — no legal move (sl. 10).

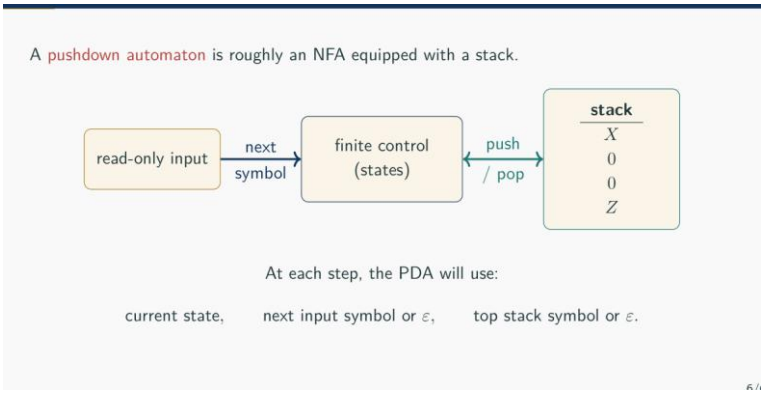


Figure 22.1 — PDA architecture: read-only input, finite control, stack (sl. 7)

2. Formal Definition (sl. 11–21)

$$P = (Q, \Sigma, \Gamma, \delta, q_0, F)$$
$$\delta : Q \times \Sigma\varepsilon \times \Gamma\varepsilon \to P(Q \times \Gamma\varepsilon)$$

Q states, Σ input alphabet, Γ stack alphabet (may contain input symbols, bookkeeping symbols, a bottom marker $ / Z), q0 start, F accepting states. (r, c) ∈ δ(q, a, b): move q → r, read a (unless ε), pop b (unless ε), push c (unless ε). δ returns a SET — the PDA is nondeterministic (sl. 16–19).

Label	Meaning
0, ε → 0	read 0, pop nothing, push 0
1, 0 → ε	read 1, pop 0, push nothing
ε, ε → $	read nothing, push the bottom marker
ε, $ → ε	remove the marker

MEMORIZE Pop/push combinations (sl. 17): X → Y replace; X → ε pop; ε → Y push; ε → ε no stack change. ε in the input position = no input read; ε is an instruction 'do nothing here', not a symbol (sl. 21).

3. Configurations and Acceptance (sl. 22–24)

Configuration (q, w, s): state, unread input, stack with top on the left. One step: if (r, c) ∈ δ(q, a, b) then (q, aw, bs) ⊢ (r, w, cs). Acceptance by final state: (q0, w, ε) ⊢* (qf, ε, s) for some qf ∈ F — the whole input must be consumed and SOME branch must end in an accepting state (stack contents irrelevant). Acceptance by empty stack is the other convention; both recognise the same class (Atif Lec 3).

4. A Complete PDA for 0^n1^n (sl. 26–28)

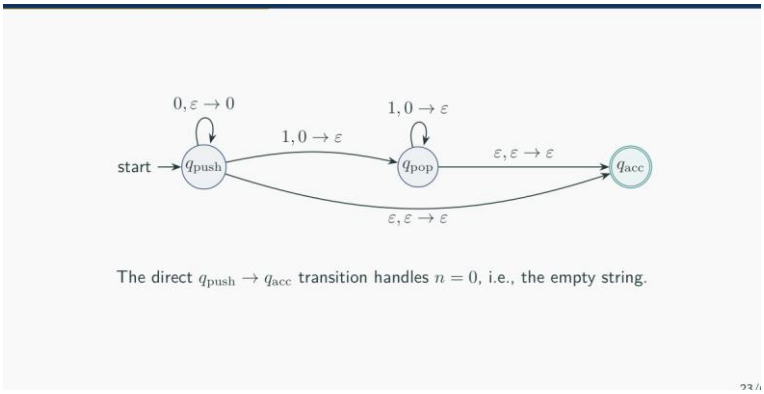


Figure 22.2 — PDA for {0^n1^n | n ≥ 0} (sl. 26)

```
(qpush, 0011, ε) ⊢ (qpush, 011, 0) ⊢ (qpush, 11, 00) ⊢ (qpop, 1, 0) ⊢ (qpop, ε, ε) ⊢ (qacc, ε, ε)
The direct qpush → qacc ε-move handles n = 0.
```

Correctness (sl. 27): every 0^n1^n pushes n and pops n markers; conversely an accepting run reads only 0s while pushing, then only 1s while popping, each 1 matching one earlier 0.

Practice — Lesson 22

(a) Constructed (concept check, sl. 69). (b) Constructed.

(a) Why does δ return a set? What does 1, 0 → ε mean? Can a PDA accept with a non-empty stack under final-state acceptance? Why must the entire input be consumed?

(b) Trace the PDA of Figure 22.2 on 001 and on 0101 and explain why each is rejected.

✓ Solution

(a) A set because the PDA may have several possible moves (nondeterminism: guess a midpoint, a branch). 1, 0 → ε: read 1 and pop a 0. Yes — final-state acceptance ignores the stack. The whole input must be read, otherwise a machine could accept after seeing only a prefix (e.g. accept 0011… after 01).

(b) 001: (qpush, 001, ε) ⊢ (qpush, 01, 0) ⊢ (qpush, 1, 00) ⊢ (qpop, ε, 0) — input finished but the only way to qacc needs the stack empty in the drawn PDA's sense (the extra 0 cannot be matched): no accepting branch. 0101: after (qpop, 01, ε) the next symbol 0 has no move in qpop — every branch dies.

Lesson 23 — Nondeterminism and Design Patterns

Slides 29–43

1. Guess the Midpoint: ww^R (sl. 30–33)

B = {ww^R}: push the first half, at the midpoint switch to matching — but no delimiter marks the midpoint. A nondeterministic ε-move guesses it; wrong guesses simply die (sl. 31).

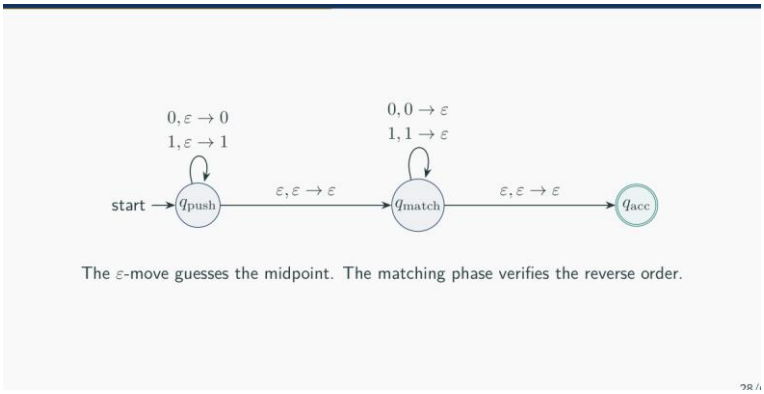


Figure 23.1 — PDA for {ww^R}: the ε-move guesses the midpoint (sl. 32)

```
(qpush, 0110, ε) ⊢ (qpush, 110, 0) ⊢ (qpush, 10, 10) ⊢ (qmatch, 10, 10) [guess] ⊢ (qmatch, 0, 0) ⊢
(qmatch, ε, ε) ⊢ (qacc, ε, ε)
```

With a delimiter, {w#w^R}, the boundary is visible and no guess is needed — a deterministic PDA suffices (sl. 34). For D = {0^i1^j2^k | i = j or j = k} the PDA branches nondeterministically into 'check i = j' or 'check j = k': **branching implements union** (sl. 35).

2. Design Patterns (sl. 37–39)

Pattern	Stack use	Typical languages
Count and match	push per symbol, pop per matching symbol	a^nb^n, a^ib^jc^k with i = j
Store and reverse	push symbols, pop and compare	w#w^R, ww^R, palindromes
Guess a boundary	ε-move to the next phase at any time	midpoints, which condition of a union, decompositions

3. Checklist and Common Mistakes (sl. 40–43)

1. What must survive an unbounded distance? What does each stack symbol mean?
2. When do you push, when pop? Is the phase boundary visible or guessed?
3. Which wrong orders must have no transition? How are ε and empty input handled?
4. Why can no string outside L reach an accepting state?

TRAP Accepting while input remains; forgetting n = 0 (ε); popping without checking the symbol; transitions that allow the wrong order; treating nondeterminism as 'one arbitrary choice' (it is 'accept if SOME branch accepts'); confusing ε with a stack symbol (sl. 41). Nondeterminism strictly increases PDA power (unlike FAs), and one stack still cannot do {0^n1^n2^n} (sl. 42–43).

Lesson 24 — Ten Worked PDA Designs and TF Problems

Slides 44–71

Convention (sl. 45): bottom marker Z pushed first; accept by final state, with the accepting move taken only when Z is visible again. Push for the part to remember, pop for the part that must match, let wrong order/count get stuck.

#	Language	Stack strategy (sl.)
1	a^nb^(2n)	push X per a; read b's in pairs, the second b of each pair pops (46–47)
2	a^ib^j, i ≤ j	push per a; each early b pops; after Z is visible extra b's are free (48–49)
3	a^ib^jc^k, i = k	push per a, ignore b's, pop per c (50–51)
4	w#w^R	push before #, pop and compare after (52–53)
5	palindromes	push, guess middle (ε for even, consume one symbol for odd), match (54–55)
6	balanced parentheses	push per (, pop per) (56–57)
7	#0 = #1	stack holds the current imbalance; opposite symbol cancels (58–59)
8	a^ib^jc^k, i = j or j = k	branch: check i = j (ignore c's) or j = k (ignore a's) (60–61)
9	a^nb^mc^md^n	push A's, then B's; pop B's on c's, A's on d's — LIFO nesting (62–63)
10	a^ib^jc^(i+j)	push per a and per b; pop per c (64–65)

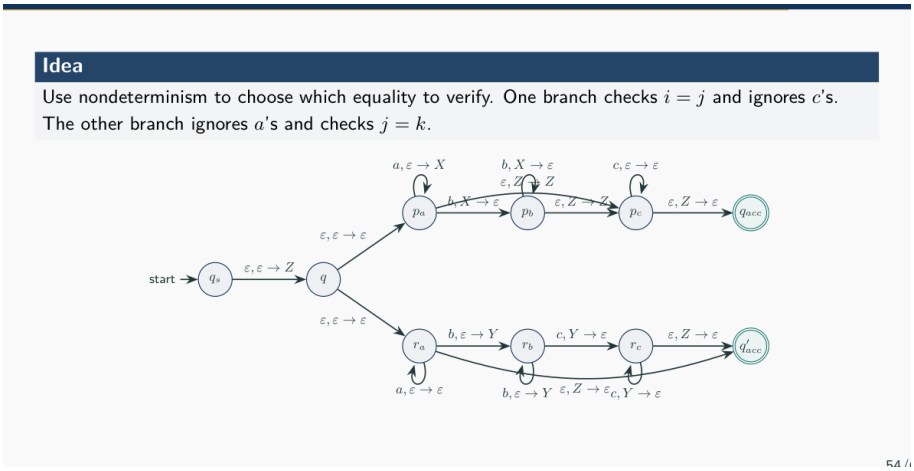


Figure 24.1 — PDA for a^ib^jc^k with i = j or j = k (sl. 60)

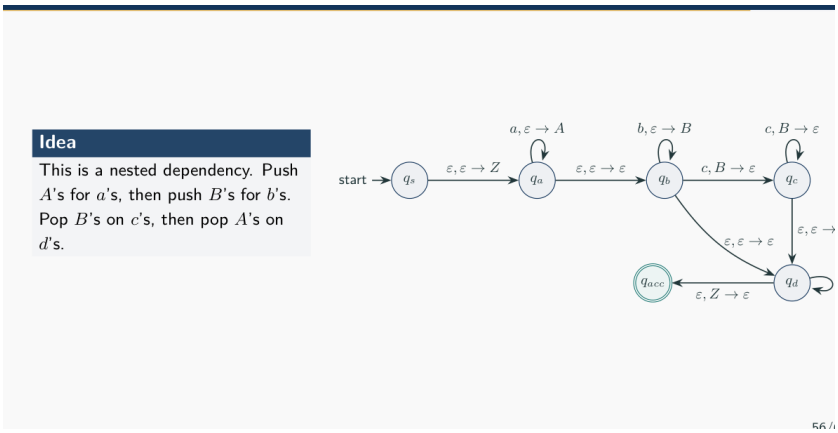


Figure 24.2 — PDA for a^nb^mc^md^n (sl. 62)

Exam Approach — PDA Design

1. Write the stack strategy in one sentence before drawing anything.

2. Draw states for phases (push / skip / pop), the Z marker move first, the Z-visible accepting move last.
3. Unions → ε-branches at the start. Unmarked boundaries → ε-moves between phases.
4. Trace one member to acceptance and say why one non-member dies.

Practice — Lesson 24

(a) Actual — 2022-23 Q8 / Jan-2020 Q8 (also tracked in Atif's tab). (b) Actual — 2023-24 Q8. (c) Actual — 2017-18 Q8. (d) Constructed — in-class problems (sl. 68).

(a) Design a PDA that recognises L = {a^ib^jc^k | i, j, k ≥ 0 and (i = j or i = k)}. Draw its transition diagram. (15)

(b) Design a PDA for L = {a^nb^mc^md^n | n ≥ 1, m ≥ 1}. Draw its transition diagram. (15)

(c) Design a PDA for L = {a^nb^nc^md^m | n ≥ 1, m ≥ 1}. (15)

(d) Stack strategies for a^nb^(2n), 0^i1^j (i ≤ j), w#w^R, 0^i1^j2^k (i = k), palindromes — is nondeterminism genuinely useful in each?

✓ Solution

(a) Strategy: branch at the start. Branch 1 (i = j): push X per a, pop X per b, when Z is visible read any c's. Branch 2 (i = k): push X per a, read b's without touching the stack, pop X per c. Accept when Z is visible at the end of a branch.

```
PDA for i = j or i = k (states: s → q → p-branch / r-branch):
     s ──ε, ε → Z──▶ q
     q ──ε, ε → ε──▶ p1
     q ──ε, ε → ε──▶ r1
    p1 ──a, ε → X──▶ p1
    p1 ──ε, ε → ε──▶ p2
    p2 ──b, X → ε──▶ p2
    p2 ──ε, Z → Z──▶ p3
    p3 ──c, ε → ε──▶ p3
    p3 ──ε, ε → ε──▶ f1
    r1 ──a, ε → X──▶ r1
    r1 ──ε, ε → ε──▶ r2
    r2 ──b, ε → ε──▶ r2
    r2 ──ε, ε → ε──▶ r3
    r3 ──c, X → ε──▶ r3
    r3 ──ε, Z → ε──▶ f2
(label: input, pop → push; ε = nothing; Z = bottom marker; accept = final state, input consumed)
```

```
Accepting computation on aabcc:
(s, aabcc, ε) ⊢ (q, aabcc, Z) ⊢ (r1, aabcc, Z) ⊢ (r1, abcc, XZ) ⊢ (r1, bcc, XXZ) ⊢ (r2, bcc, XXZ) ⊢
(r2,
cc, XXZ) ⊢ (r3, cc, XXZ) ⊢ (r3, c, XZ) ⊢ (r3, ε, Z) ⊢ (f2, ε, ε)
```

aabcc has i = k = 2: branch 2 accepts. aabbc: branch 1 accepts. aabc: both branches die (i ≠ j, i ≠ k).

(b) Strategy: nested dependency — push A per a; the first b pushes B on top of A's; pop B per c; pop A per d. LIFO forces all c's to match b's before any d meets an a (sl. 62–63). n, m ≥ 1 is enforced because the first b must see an A and the first c must see a B.

```
PDA for a^nb^mc^md^n, n, m ≥ 1:
     s ──ε, ε → Z──▶ qa
    qa ──a, ε → A──▶ qa
    qa ──b, A → BA──▶ qb
    qb ──b, ε → B──▶ qb
    qb ──c, B → ε──▶ qc
    qc ──c, B → ε──▶ qc
    qc ──d, A → ε──▶ qd
    qd ──d, A → ε──▶ qd
    qd ──ε, Z → ε──▶ acc
(label: input, pop → push; ε = nothing; Z = bottom marker; accept = final state, input consumed)
```

```
Accepting computation on aabbccdd:
(s, aabbccdd, ε) ⊢ (qa, aabbccdd, Z) ⊢ (qa, abbccdd, AZ) ⊢ (qa, bbccdd, AAZ) ⊢ (qb, bccdd, BAAZ) ⊢
(qb, c
cdd, BBAAZ) ⊢ (qc, cdd, BAAZ) ⊢ (qc, dd, AAZ) ⊢ (qd, d, AZ) ⊢ (qd, ε, Z) ⊢ (acc, ε, ε)
```

(c) Strategy: sequential blocks — push X per a, pop X per b; when Z reappears the first c pushes a new counter (Y above Z), further c's push Y, d's pop Y; accept when Z is visible after the d's.

```
PDA for a^nb^nc^md^m, n, m ≥ 1:
     s ──ε, ε → Z──▶ qa
    qa ──a, ε → X──▶ qa
    qa ──b, X → ε──▶ qb
    qb ──b, X → ε──▶ qb
    qb ──c, Z → YZ──▶ qc
    qc ──c, ε → Y──▶ qc
    qc ──d, Y → ε──▶ qd
    qd ──d, Y → ε──▶ qd
    qd ──ε, Z → ε──▶ acc
(label: input, pop → push; ε = nothing; Z = bottom marker; accept = final state, input consumed)
```

(d) a^nb^(2n): push X per a, pop every second b — deterministic. 0^i1^j, i ≤ j: push per 0, pop per 1, then allow extra 1s on Z — deterministic. w#w^R: deterministic (the # marks the middle). 0^i1^j2^k, i = k: the end of the 0-block is visible, deterministic in principle. Palindromes: nondeterminism is essential — the middle must be guessed (and whether the length is odd).

Quick Notes — PDA Design

Revision points for Lessons 22–24.

L22 · PDA

- P = (Q, Σ, Γ, δ, q0, F), δ : Q × Σε × Γε → P(Q × Γε). Label a, b → c = read a, pop b, push c.
- Configuration (q, w, s), top on the left. Accept: input consumed, some branch in F.
- 0^n1^n: push per 0, pop per 1; ε-move qpush → qacc for n = 0.

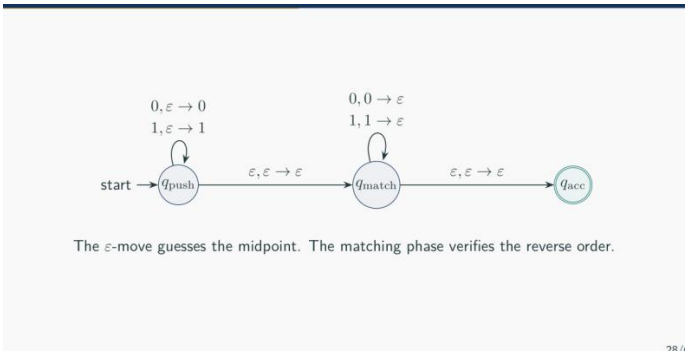
L23 · Nondeterminism

- ww^R: ε-move guesses the midpoint; w#w^R needs no guess.
- Union = branch at the start. Patterns: count/match, store/reverse, guess boundary.
- Mistakes: accept with input left, forget ε, pop without checking, wrong order allowed.

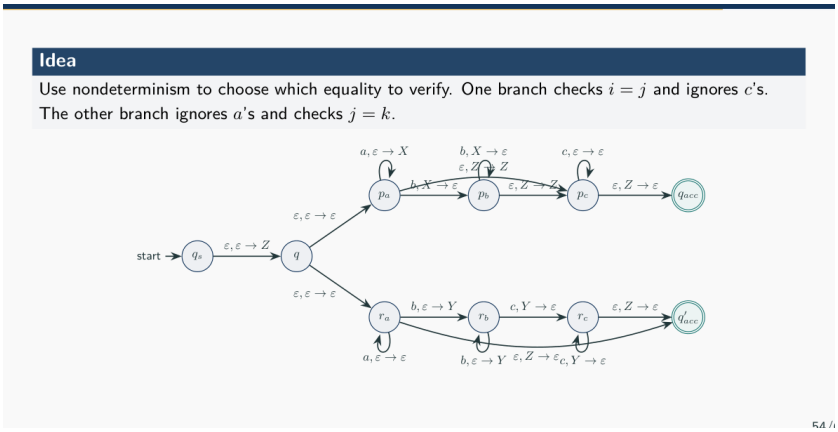
L24 · Designs

- i = j or i = k: branch 1 pop on b's (c free); branch 2 b's free, pop on c's.
- a^nb^mc^md^n: push A's then B's; pop B on c, A on d. a^nb^nc^md^m: two counters in sequence.
- #0 = #1: stack stores the imbalance. a^ib^jc^(i+j): push per a and b, pop per c.

Key Diagrams to Reproduce



PDA for ww^R (sl. 32)



PDA for i = j or j = k (sl. 60)